World Electric Vehicle Journal

MDPI

Article

A Novel Approach to Autonomous Driving Using Double Deep Q-Network-Bsed Deep Reinforcement Learning

Ahmed Khlifi [1,2,*], Mohamed Othmani [2,3,*] and Monji Kherallah [2,4]

[1] National Engineering School of Sfax, University of Sfax, Sfax 3029, Tunisia
[2] Advanced Technologies for the Environment and Smart Cities ATES, Faculty of Sciences of Sfax, University of Sfax, Sfax 3029, Tunisia; monji.kherallah@fss.usf.tn
[3] Faculty of Sciences of Gafsa, University of Gafsa, Gafsa 2112, Tunisia
[4] Faculty of Sciences of Sfax, University of Sfax, Sfax 3029, Tunisia
* Correspondence: ahmed.khelifi@issatgf.u-gafsa.tn (A.K.); mohamed.othmani@fsgf.ugaf.tn (M.O.)

Abstract: Deep reinforcement learning (DRL) trains agents to make decisions by learning from rewards and penalties, using trial and error. It combines reinforcement learning (RL) with deep neural networks (DNNs), enabling agents to process large datasets and learn from complex environments. DRL has achieved notable success in gaming, robotics, decision-making, etc. However, real-world applications, such as self-driving cars, face challenges due to complex state and action spaces, requiring precise control. Researchers continue to develop new algorithms to improve performance in dynamic settings. A key algorithm, Deep Q-Network (DQN), uses neural networks to approximate the Q-value function but suffers from overestimation bias, leading to suboptimal outcomes. To address this, Double Deep Q-Network (DDQN) was introduced, which decouples action selection from evaluation, thereby reducing bias and promoting more stable learning. This study evaluates the effectiveness of DQN and DDQN in autonomous driving using the CARLA simulator. The key findings emphasize DDQN's advantages in significantly reducing overestimation bias and enhancing policy performance, making it a more robust and reliable approach for complex real-world applications like self-driving cars. The results underscore DDQN's potential to improve decision-making accuracy and stability in dynamic environments.

Keywords: deep reinforcement learning; neural networks; autonomous driving; Deep Q-Network; Double Deep Q-Network; CARLA simulator

check for updates

Academic Editor: Michael Fowler

Received: 11 January 2025
Revised: 21 February 2025
Accepted: 25 February 2025
Published: 1 March 2025

Citation: Khlifi, A.; Othmani, M.; Kherallah, M. A Novel Approach to Autonomous Driving Using Double Deep Q-Network-Bsed Deep Reinforcement Learning. *World Electr. Veh. J.* **2025**, *16*, 138. https://doi.org/10.3390/wevj16030138

Copyright: © 2025 by the authors. Published by MDPI on behalf of the World Electric Vehicle Association. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Autonomous driving has been a topic of interest for many years, with various advancements being made to improve its safety and efficiency [1]. However, developing a fully autonomous system presents several challenges, such as the need for real-time decision-making and adaptability to changing environments. One of the most promising technologies used in this field is deep reinforcement learning, a type of Machine Learning that uses neural networks and reward-based training to enable decision-making [2]. Despite these promising advancements, deep reinforcement learning also presents challenges and ethical implications [3]. For example, it raises concerns about responsibility if an autonomous vehicle causes an accident. Determining fault involves assessing whether the manufacturer, the software developer, or the vehicle owner is responsible. Moreover, there are concerns about the impact of autonomous driving on employment and the economy.

Nonetheless, the potential benefits of DRL in this field are vast, and the technology continues to evolve. For instance, developing more advanced neural networks and integrating other technologies, such as computer vision and natural language processing, can further enhance the decision-making process of autonomous systems.

Deep reinforcement learning enhances autonomous driving systems by allowing agents to learn from their mistakes and improve decision-making over time [4]. For example, an agent can receive a reward for staying within the speed limit and a penalty for breaking it [5]. This reward-based training helps the agent learn from past experiences, making adapting to different road and weather conditions possible [6]. As a result, DRL has the potential to revolutionize autonomous driving by enhancing the system's ability to respond effectively to diverse situations.

This work aims to assess autonomous driving tasks in urban environments using DQN agents. To achieve this, several approaches based on DQN agents will be investigated. The DQN agent learns a policy, and a set of behaviors, for lane-following tasks by applying visual and driving characteristics obtained from in-vehicle sensors and a trajectory planner based on a model [7–9]. This method comprehensively analyzes how deep reinforcement learning is transforming autonomous driving technology.

An end-to-end autonomous system based on the Deep Q-learning algorithm offers several advantages over traditional approaches [10]. Its simplicity lies in the seamless integration of perception, prediction, and planning into a unified model that can be trained together. Our study proposes an RL-based autonomous driving system, emphasizing more informative exploration and improved reward signaling. We will evaluate the performance of this system in urban environments using the DDQN approach combined with Long Short-Term Memory (LSTM) [11].

We will develop an intelligent driving agent capable of navigating complex environments along predetermined routes [12,13], such as those in the CARLA simulator, to validate our approach [14]. We will also analyze various design decisions to determine the best configurations for training autonomous driving agents using reinforcement learning. Additionally, we will demonstrate the training methods that can significantly impact the performance of a deep RL agent. Finally, we will validate our approach in various challenging traffic scenarios and show that our method outperforms previous state-of-the-art techniques.

2. Overview of Deep Reinforcement Learning in Autonomous Driving

2.1. Reinforcement Learning

RL is a key artificial intelligence technique for tackling complex problems like robotics [15], industry automation, natural language processing, and autonomous driving. In autonomous driving, RL trains vehicles to make decisions in dynamic environments. The vehicle, the agent, interacts with its surroundings, including roads, traffic, and pedestrians, by taking actions such as steering, accelerating, and braking. Each action earns rewards or penalties based on its safety and effectiveness, guiding the vehicle to improve over time.

Autonomous driving with RL involves balancing exploration, i.e., trying new maneuvers, and exploitation, i.e., using proven strategies, to maximize rewards [16]. Simulations play a vital role in training RL models safely before real-world deployment, addressing the challenges of complex, dynamic environments. Unlike supervised learning, where explicit examples are provided, RL relies on trial and error to optimize actions, using feedback to refine strategies and ensure safe, efficient performance. The general agent–environment interaction process in RL is shown in Figure 1.

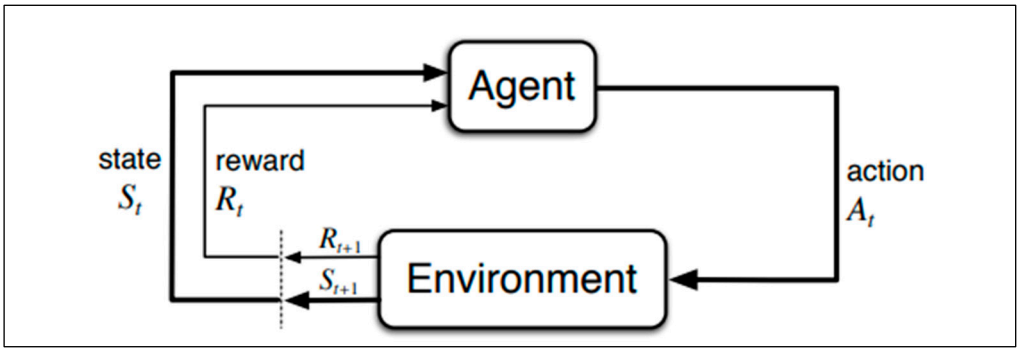

Figure 1. The agent–environment interaction in reinforcement learning, reprinted from Ref. [17].

The learning and decision-making in RL critically involve an interaction between the agent and the environment. A_t a given time step t, the environment is in a certain state, S_t, from which the agent takes an action, A_t, according to its policy, which has assigned a strategy to map states to actions. The execution of action A_t causes the environment to transition into the next state, S_{t+1}, and gives the agent a reward R_t. The agent then observes this reward and new state, S_{t+1}. At the next time step, t+1, an updated action, A_{t+1}, is taken by the agent in response to its updated state. It goes on repeatedly that an agent selects an action, receives a reward, and observes the next state. It tries to maximize rewards over time, which can be measured regarding a sum of rewards, discounted rewards, or another metric concerning long-term benefit. This then creates an iterative feedback loop through which the agent can improve its policy and further develop its strategy in making decisions [18].

2.2. Deep Reinforcement Learning

DRL has become a key approach for training autonomous vehicles to make complex decisions in dynamic environments [19]. By combining reinforcement learning and deep learning, DRL allows vehicles to navigate, plan, and control effectively while handling tasks like lane changes, intersections, and obstacle avoidance. Training often occurs in simulated environments, enabling the safe exploration and refinement of strategies. Using reward systems, DRL optimizes decision-making by encouraging safe behaviors and penalizing unsafe actions [20]. Techniques like Convolutional Neural Networks process sensory data to enhance perception and planning. Despite its promise, challenges remain, such as ensuring generalization to new scenarios, improving safety mechanisms, and addressing computational efficiency. DRL's hierarchical structures help tackle complex traffic scenarios, improving speed, trajectory, and collision avoidance, while algorithms like DQN are evaluated for maneuver planning. Simulations significantly reduce real-world testing needs, minimizing risks and enhancing algorithm performance.

2.3. Deep Q-Networks (DQNs)

DQNs combine Q-learning with deep neural networks to enable reinforcement learning agents to handle high-dimensional state spaces. By approximating the Q-value function using a neural network, DQNs allow agents to learn optimal policies directly from raw inputs, such as images or sensor data [8]. This approach has been successfully applied in various domains, including playing video games, robotics, and autonomous driving, showcasing its ability to solve complex decision-making problems.

2.4. Double Deep Q-Networks (DDQNs)

DDQN is an enhancement to the original DQN algorithm designed to address the overestimation bias in Q-learning, which arises when the same network is used to select and

evaluate actions, leading to inflated Q-values [21]. Unlike DQN, DDQN employs two separate networks: an online network for action selection and a target network for Q-value evaluation. This decoupling reduces bias and improves learning stability and Q-Network (DDQN)-based deep reinforcement learning (DRL) algorithms to enhance decision-making and control. Several novel approaches have emerged, each addressing enhancing performance in complex environments by ensuring more accurate Q-value updates. Thus, recent advancements in autonomous driving have leveraged Double Deep specific challenges. Notable approaches include Dueling DDQN for lane-keeping, which improves stability by separately estimating state value and advantage functions [22], and Game-Theoretic DDQN for Intersection Control, enabling cooperative vehicle interactions [23]. The Double Broad Q-Network (DBQN) enhances overtaking decisions using a broad learning system [24], while Hierarchical Dueling DDQN improves sparse-reward learning for complex driving tasks [25]. Lastly, LK-TDDQN applies transfer learning to adapt lane-keeping strategies across environments [26]. These innovations demonstrate the versatility of DDQN-based DRL in autonomous driving.

3. Benchmarking—Urban Driving Simulator

Simulation is a cost-effective and safe alternative to real-world testing for autonomous systems. It allows developers to quickly create and evaluate prototypes, iterate on designs, and explore various scenarios. Simulators also provide precise performance measurement tools, enabling the in-depth analysis and optimization of algorithms. This approach accelerates development and enhances safety by allowing for comprehensive testing in controlled virtual environments.

CARLA (Car Learning to Act) is an open-source simulator for urban driving specifically designed for the study of autonomous vehicles [27]. It was built from the ground up to simplify the creation, training, and evaluation of autonomous driving systems in urban environments. CARLA provides tools to perform detailed simulations and evaluate system performance. In addition to its open-source code and protocols, CARLA offers open digital assets, such as urban layouts, buildings, and vehicles, that users can use independently. The deep customization of sensor sets and environmental conditions is possible through the platform, enabling accurate simulation [28,29].

Each town in CARLA possesses its own unique characteristics. Towns (Figure 2) was utilized as the primary platform for training our agent and the performance evaluation.

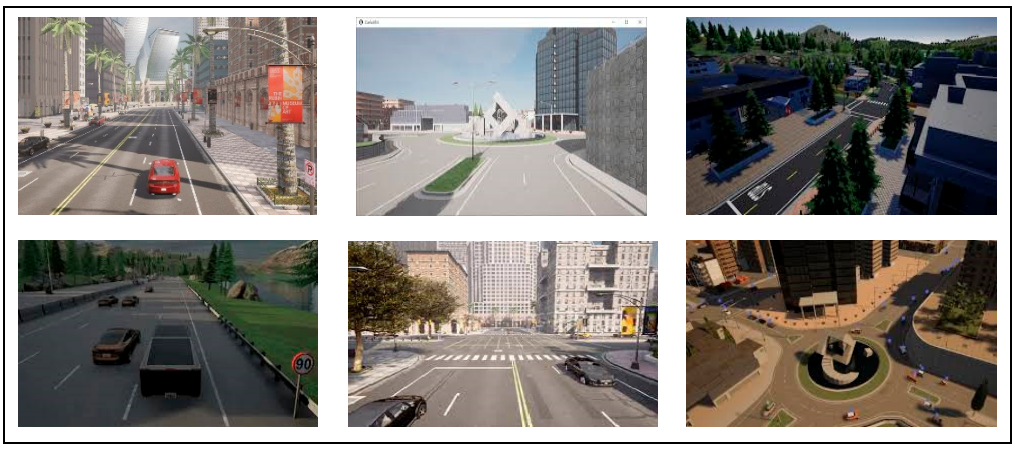

Figure 2. CARLA driving simulator screenshots.

4. Related Work

In recent years, deep reinforcement learning (DRL) has achieved great success in the field of autonomous vehicles. Due to this great success, several researchers have chosen to use it in their research. Their main challenge is to develop an intelligent agent that can on the one hand avoid obstacles and mitigate collisions of autonomous vehicles and on the other hand correct errors from autonomous driving pipeline tasks such as decision-making and motion planning. In this context, several approaches have used the DQN-based DRL algorithm that has demonstrated great effectiveness in ensuring safe navigation in various simulated dynamic environments, namely CARLA [30].

Elallid et al., 2022 present an approach that uses the CARLA simulator that is designed to imitate real-world streets to train and validate autonomous vehicles [9]. This employed method for controlling an AV in complicated environments is based on a DQN model. The car is equipped with a front-facing camera to take real-time pictures. The captured photos, which were originally 640 × 480 pixels in RGB, are first converted to grayscale levels and then resized to 192 × 256 pixels. These processed images are then passed through two dense couches with 512 and 256 neurons, enabling the model to generate 190 alternative actions. In the CARLA environment, a 389 m course with right turns and intersections was designed. Ten pedestrians and five vehicles were added to make the environment more dynamic and realistic. The model was trained for 5000 episodes, with a mini-batch size of 16, using a repetition memory system to learn from past experiences. The results show that the model learns effectively across episodes: average rewards increase as the success rate of actions improves, and the collision rate gradually decreases until reaching almost zero. This demonstrates that the AV learns to avoid accidents with other vehicles and pedestrians present in the environment, ensuring safe driving.

Hossain et al. (2023) proposed a model based on a deep neural network to implement the DQN algorithm, used to approximate the Q-value function [31]. This function evaluates the quality of each possible action in each state. The agent, representing the autonomous car, interacts with a simulated environment, where it receives observations, chooses actions, and learns to maximize cumulative rewards over time. The model architecture consists of several layers designed to process the observations of the environment and produce the Q-values associated with each possible action. The observations, constituting the input space, are represented by a 5 × 5 array describing the vehicles in the vicinity of the autonomous car. Each row of the table corresponds to a vehicle, with columns indicating the following characteristics: position (x, y) and speed (Vx, Vy). This information is processed by the neural network to determine the optimal action. The neural network is a Multi-Layer Perceptron (MLP) comprising several fully connected layers. These layers learn to identify complex relationships between vehicles, such as neighborhood and relative speed, to assess the quality of possible actions. Activation functions such as ReLU (Rectified Linear Unit) are used to introduce non-linearity and enable the model to better approximate the Q function. As an output, the network produces a vector of Q-values for each possible action in the environment. The action space comprises five possible actions: change lanes to the left (LANELEFT), stay put (IDLE), change lanes to the right (LANERIGHT), accelerate (FASTER), and slow down (SLOWER). Each Q-value represents the quality of the associated action in the current state of the environment. The agent then selects the action with the highest Q-value, corresponding to that which maximizes the expected cumulative reward. The reward function is designed to encourage fast and safe driving behavior. Thus, the agent receives a reward of 0.1 points when staying in the right lane and a reward of 0.4 points when maintaining a high speed. On the other hand, there is no specific penalty or incentive for lane changes, which therefore do not directly affect the reward (0 points).

For each action performed, the agent receives these rewards, which incentivizes it to adopt a behavior that maximizes speed while avoiding collisions.

Tammewar et al., 2023, studied the improvement in autonomous driving performance using DRL [32]. The approach involves training a simulated vehicle to navigate autonomously on a racing track using the DQN algorithm. The system uses the CarRacing-v2 simulator which provides a top view of a randomly generated track. The vehicle receives visual information from a front camera and interacts with the environment via actions. The built model receives input images from the front camera, represented as RGB pixels (96 × 96 pixels in this case). The images are subsequently converted to grayscale to reduce computational complexity and focus the model's attention on important structural aspects of the image, such as the contours of the track. These images are used to capture information about the vehicle's environment (speed, direction, etc.). The inputs are then processed using CNNs to extract relevant features. To capture temporal dependencies and understand the dynamics of vehicle movement across images, a recurrent neural network (LSTM) is used after the CNN layers. The goal of this part is to allow the model to retain information over multiple time steps and to adjust its actions based on past trajectories. Rewards are assigned based on the coverage of track tiles, while penalties are applied when the vehicle goes off track. As an output, the model chooses among possible actions (acceleration, braking, steering) based on the extracted features. In the continuous version, the actions are represented by three parameters: direction (from −1 to 1), acceleration (from 0 to 1), and braking (from 0 to 1). These actions aim to help the vehicle navigate the track while maximizing the reward obtained. The results show that the DQN algorithm with epsilon decay (ε-decay) performed well and provided excellent stability and efficiency as well as cumulative scores over episodes for the autonomous navigation task.

In these approaches, although the agents were tested in various simulation environments, their performance may not generalize to other real-world environments with very different driving scenarios. Likewise, the policies learned may be too specialized for the specific conditions of the simulation (traffic, weather, road infrastructure), which limits their applicability in varied real-life situations.

5. Materials and Methods

The main methodology of this work is to introduce a novel approach based on deep reinforcement learning, which will enable a car to drive autonomously in a virtual environment. Since the system is based on a computer-generated environment, the CARLA_0.9.13 simulator for autonomous cars is the environment used. Our research focused on the impact of various hyperparameters. The effects on the convergence and robustness of learning were studied using learning rates for each model. Our main performance measures were the consistency of training each model over a given number of episodes, with episode reward and learning stability as the primary measures. A controlled and reproducible comparison between models was facilitated by this approach, ensuring that any observed performance disparities were related to the intrinsic characteristics of the models and the chosen hyperparameters.

In this work, we propose a novel architecture that combines CNNs, LSTM, and Deep Q-learning with a DDQN to tackle reinforcement learning tasks [33–35]. This hybrid model leverages CNNs to extract spatial features from image data and LSTMs to capture the temporal dependencies in sequential data, making it particularly well suited for environments where inputs are image sequences (e.g., video frames).

The integration of DDQN enhances the reinforcement learning component by addressing the overestimation bias common in standard Q-learning [14]. This allows the model to make more stable and accurate decisions in dynamic and complex environments. To

prevent overfitting, dropout is applied within the convolutional layers, which is crucial when working with high-dimensional input data and limited training samples.

The architecture is modular and flexible, enabling the configuration of key parameters such as the number of LSTM layers, hidden units, and CNN filters to adapt to various task complexities. By combining the strengths of CNNs, LSTMs, and DDQN, this approach presents a robust and efficient solution for reinforcement learning tasks involving sequential image data. The model's structure is shown in Figure 3.

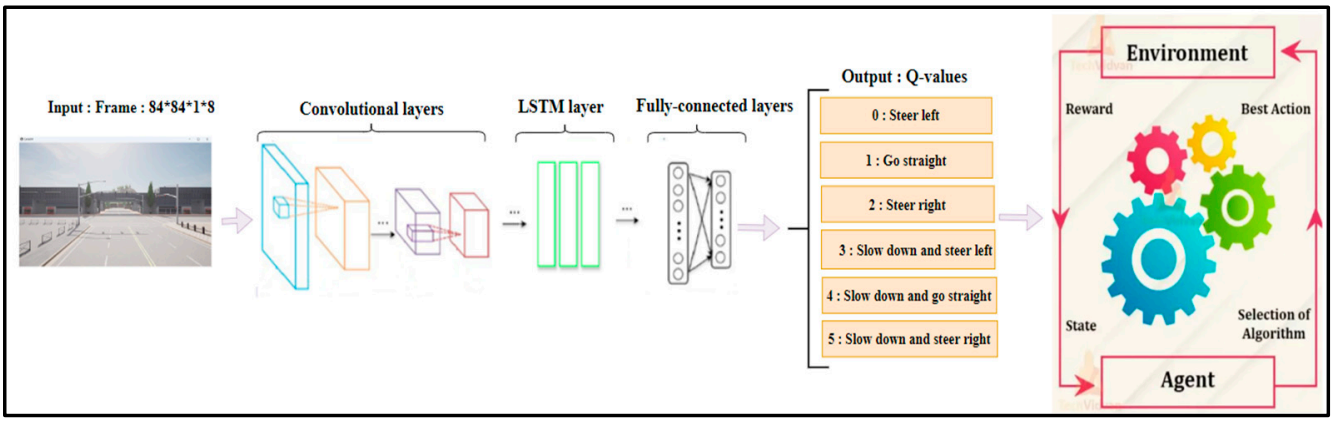

Figure 3. Proposed model architecture.

5.1. Model Architecture and State Space

In our scenario, we process a stack of four RGB images captured by the front camera of the autonomous vehicle (AV). Initially, each image has 640 × 480 × 3 pixels. We resize them to 84 × 84 × 3 pixels, then convert them to grayscale. This transformation yields a new state, denoted as S_t, with dimensions of 84 × 84 × 1, which are fed into the input of the neural network. The model combines convolutional layers with an LSTM, followed by fully connected layers for final predictions. By correcting the overestimation of action values that can occur in the original algorithm, DDQN enhances the conventional Q-learning algorithm.

The Q-values of each action in each state, which stand for the anticipated future benefit of performing that action in that state, are learned by the algorithm in typical Q-learning. However, when function approximation is used, which occurs frequently in large state spaces, these Q-values might become overstated. As a result, less-than-ideal decisions may be made.

By creating a second network that is used to choose the actions to be executed, DDQN resolves this problem. While the secondary network is used to assess the Q-value of the selected action, the primary network is utilized to estimate the Q-values of each action in each state. To reduce the overestimation of Q-values, the idea is to decouple the selection and evaluation of activities.

Our architecture is mainly based on multiple layers such as convolutional layers. The number of convolutional layers is four, as shown in the architecture figure. The first convolutional layer applies 32 convolutional filters with a kernel size of 8 × 8 and a stride size of 4. It reduces the spatial dimensions of the input while extracting the initial feature maps. A dropout with a probability of 0.4 is applied after the first convolutional layer to mitigate overfitting. The second convolutional layer uses 64 convolutional filters with a kernel size of 4 × 4 and a stride size of 2 to extract more features and reduce the spatial dimensions. A dropout with a probability of 0.4 is applied after the second convolutional layer. For the third convolutional layer, 64 convolutional filters with a kernel size of 3 × 3 and a stride size of 1 are used to add complexity to feature extraction. Two max pooling layers, each with a kernel size and stride of 2, are used to further downsample the feature

maps and capture the most salient features. A fourth convolutional layer with 64 filters, a kernel size of 3 × 3, and a stride of 1 is used to refine the features.

As shown in Figure 4, after the convolution and pooling operations, the spatial dimensions are flattened to prepare the data for the LSTM layer. The tensor is reshaped from (batch_size * seq_len, c, h, w) to (batch_size, seq_len, −1), where −1 automatically calculates the size of the flattened feature. Afterward, an LSTM layer captures the temporal dependencies in the sequence of image frames. It consists of hidden units lstm_hidden_size and layers num_lstm_layers. input_size is the flattened size of the output of the convolutional layers. Next, we find the first fully connected layer that transforms the LSTM output into a 512-dimensional vector. Finally, the last fully connected layer maps the 512-dimensional vector to a vector of size num_actions, representing the Q-values for each possible action in the reinforcement learning task.

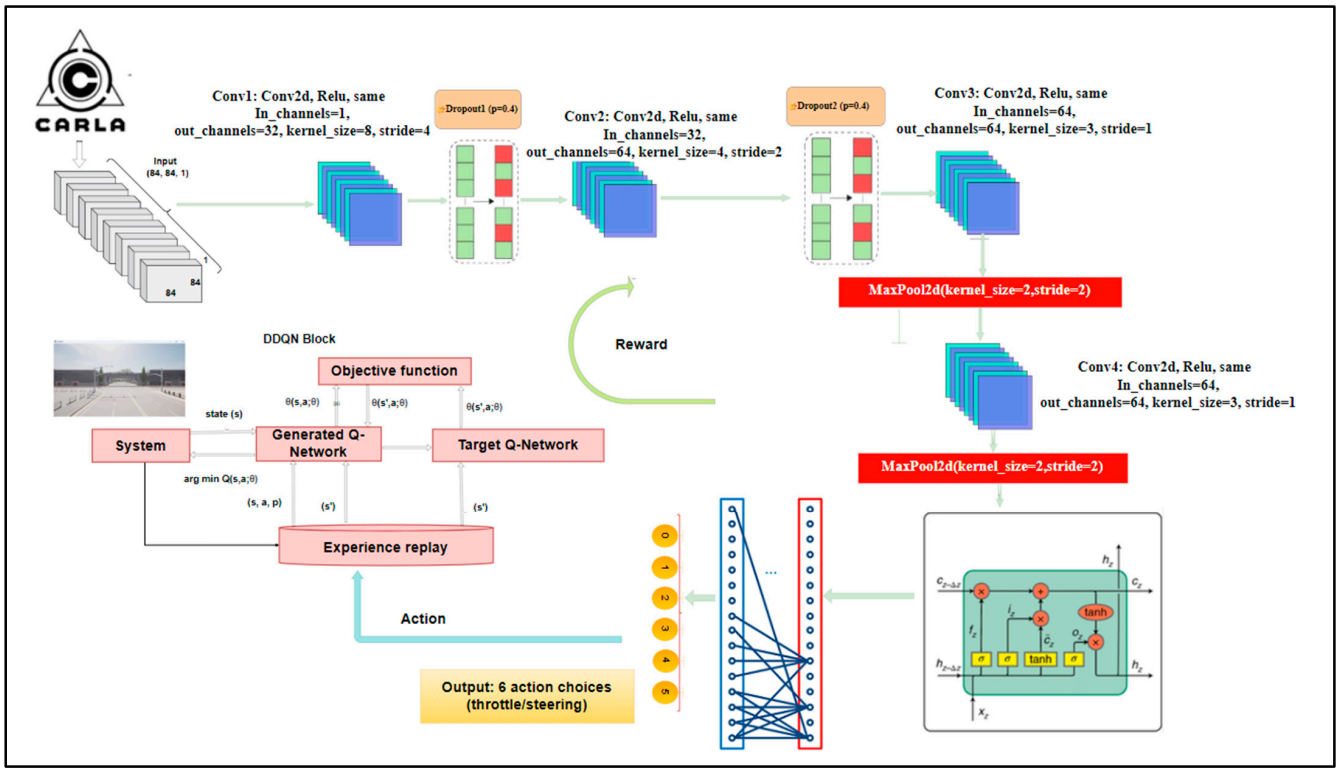

Figure 4. The proposed CNN-LSTM-DDQN architecture.

For the forward pass, the input x is processed through the convolutional layers, followed by the LSTM layer to capture temporal dependencies, and finally through the fully connected layers to produce the Q-values. In this model, we aim to design a robust architecture for reinforcement learning tasks involving image sequences, exploiting the strengths of convolutional and recurrent neural networks to process and learn from complex temporal data. The architecture of our system is a DDQN with the inputs and outputs shown in Table 1:

Table 1. Architecture of proposed system.

Layer	Input Dimensions	Output Dimensions	Activation	Dropout	Notes
Convolutional 1	84 × 84 × 1	32 × 42 × 32	ReLU	Yes (0.4)	8 × 8 kernel, stride 4
Convolutional 2	32 × 42 × 32	64 × 21 × 64	ReLU	Yes (0.4)	4 × 4 kernel, stride 2
Convolutional 3	64 × 21 × 64	64 × 10 × 64	ReLU	No	3 × 3 kernel, stride 1
Max Pooling 1	64 × 10 × 64	64 × 5 × 64	N/A	No	2 × 2 kernel, stride 2
Convolutional 4	64 × 5 × 64	64 × 4 × 64	ReLU	No	3 × 3 kernel, stride 1
Max Pooling 2	64 × 4 × 64	64 × 2 × 64	N/A	No	2 × 2 kernel, stride 2
LSTM	64 × 2 × 64 (flattened)	256 hidden units	N/A	No	1 layer
Fully Connected 1	256	512	ReLU	No	Dense layer
Fully Connected 2	512	6	N/A	No	Action output layer

5.2. Reward Function

The reward function in the provided code is structured to guide the agent's behavior based on its interactions with the environment. The primary factors influencing the reward are collisions, the duration of the episode, and whether the agent avoids obstacles during its driving task. The reward function in this method is based on several specific scenarios, described in our approach. Here are the details of the different cases in this driving simulation: Firstly, in the event of a collision, detected by the presence of items in the collision_hist list, the reward is set to −20, reflecting a severe penalty for this incident. The episode immediately ends with done equal to True, marking the end of the episode following an accident. Then, if no collision occurs and the car continues to run smoothly, a + 5 reward is awarded. This encourages collision-free driving, and the episode continues with done equal to False, allowing the agent to continue the episode without interruption. Finally, if the episode lasts longer than 30 s, a significant reward of 250 is awarded. The episode ends with a done equal to True, reflecting a bonus for driving through the entire episode without major incident. This scenario offers an additional incentive to maintain prolonged, safe driving. In summary, this reward structure strongly penalizes collisions while promoting continuous, accident-free driving, enabling the agent to learn to avoid collisions and drive stably. Here is a summary of the reward function in pseudo-code form:

$$reward = \begin{cases} -20 & if\ collision \\ 250 & if\ maximum\ time\ reached \\ +5 & otherwise \end{cases} \tag{1}$$

This encourages the agent to avoid collisions and continue driving until the episode ends.

5.3. Mounted Sensors and Hyperparameters

Sensors mounted on autonomous vehicles play a crucial role in their ability to perceive and understand their environment in real time to make safe and efficient decisions. To achieve this, several types of sensors are integrated into autonomous vehicles, each providing specific data to analyze different aspects of the environment. The following table introduces the types of data commonly captured by these sensors and their respective functions:

Table 2 outlines the two sensors used in our simulation, with details on their attributes, functions, and roles in controlling the autonomous vehicle.

Table 2. Data for sensors mounted on autonomous vehicle.

Sensor	Usage	Specific Attributes	Location of the Vehicle	Function
RGB Camera	Images are resized to 84 × 84 pixels and converted to grayscale to be processed by our model. Used for autonomous decision-making.	Resolution: 640 × 480 pixels (modifiable). Field of View (FOV): 110 degrees.	Mounted at the front, coordinates: x = 2.5, z = 0.7.	Captures color images for the visual perception of the environment.
Collision Sensor	Detects vehicle collisions with the environment and logs these events.	Logs collisions in a collision_hist list. Negative reward assigned in the case of collision, with a penalty of −20 points.	Attached to the vehicle (exact position unspecified).	Used to evaluate safety during training, penalizing unsafe behavior and preventing crashes.

It is also necessary to define the set of various hyperparameters that serve as outside configuration variables utilized to control model training. These hyperparameters determine important model properties like design, learning rate, and complexity.

Table 3 below details the hyperparameters::

Table 3. Hyperparameters our model.

Hyperparameters	Value	Description
SHOW_PREVIEW	FALSE	Controls whether the front camera preview is shown during the simulation.
IM_WIDTH	640	Width of the camera image.
IM_HEIGHT	480	Height of the camera image.
SECONDS_PER_EPISODE	30	Maximum time (in seconds) for each episode in the environment.
MIN_REWARD	−20	Minimum reward threshold to consider for episode termination.
STEER_AMT	1	Steering amount (how much the vehicle turns when an action is taken).
num_frames	8	Number of image frames to stack for input to the neural network.
Gamma	0.99	Discount factor is used in Q-learning to calculate the future expected rewards.
Batch_Size	32	Number of transitions to sample from the replay buffer in each training iteration.
Buffer_Size	5,000,000	Maximum size of the replay buffer (number of stored transitions).
Min_Replay_Size	100,000	Minimum number of transitions to collect before starting training.
Episodes	2000	Total number of episodes to run the training loop.
Epsilon	1	Initial exploration rate for ε-greedy policy (controls how often random actions are taken).
min_epsilon	0.0001	Minimum value to which epsilon can decay.

Table 3. *Cont.*

Hyperparameters	Value	Description
Decay	(min_epsilon/ epsilon)**(1/ episodes)	Decay factor for reducing epsilon after each episode.
lstm_hidden_size	256	Number of hidden units in the LSTM layer.
num_lstm_layers	1	Number of layers in the LSTM.
Optimizer	Adam	Optimization algorithm used for training the neural network.
learning_rate	5.00×10^{-4}	Learning rate for the Adam optimizer.
Dropout	0.4	Dropout rate used in the CNN layers to prevent overfitting.
target_net_update_freq	Every 4 episodes	Frequency of updating the target network with the weights from the online network.
max_pool_kernel_size	2	Kernel size for max pooling layers in the CNN.
reward_success	250	Reward is assigned when the vehicle completes an episode without collision within the time limit.
reward_collision	−20	Reward penalty is assigned when a collision occurs.
reward_step	5	Reward for each successful step taken without a collision.

For the **, designates the mathematical power.

These space hyperparameters control various aspects of the neural network architecture, reinforcement learning algorithm, and training dynamics.

5.4. Action Space

Our model inputs forward-facing RGB camera images from the CARLA simulator. Each image is converted to grayscale and resized to 84 × 84 for processing. The output of our system is actions: there are 6 possible actions (combinations of steering and throttle). In the CARLA simulator environment, the AV interacts with its environment using four main control commands: steer left, steer right, go straight, and slow down. These commands are represented as integer values in the range 0 to 5. DDQN is a discrete DRL algorithm; the agent must make discrete action choices as per Table 4. The action is to be taken by car. The possible actions are shown in the following table:

Table 4. Actions and their corresponding values.

Actions	Control Commands
0	Steer left
1	Go straight
2	Steer right
3	Slow down and steer left
4	Slow down and go straight
5	Slow down and steer right

Algorithm

The provided algorithm presents a clear and concise implementation of the DDQN for autonomous navigation:

Initialize CARLA environment and sensors

For each episode:

Reset environment and variables (collisions, camera, etc.)

As long as the episode has not ended:

Choose an action (epsilon-greedy)

Execute action in environment

Obtain new observation, reward, collision status, etc.

Add transition to buffer

If buffer is sufficient:
Sample a batch of transitions
Calculate Q-targets
Calculate loss between current Q-values and targets
Update model weights by minimizing loss
If episode is over or time is up, stop
Every N episodes:
Update target network with online network weights
Record model weights
Track rewards and display results

6. Results and Discussion

The suggested approach was validated using the free autonomous driving simulation program CALRA [3,9,34]. Several training simulations were carried out in the Town 05 environment using CARLA 0.9.13. The simulation environment is shown in Figure 5.

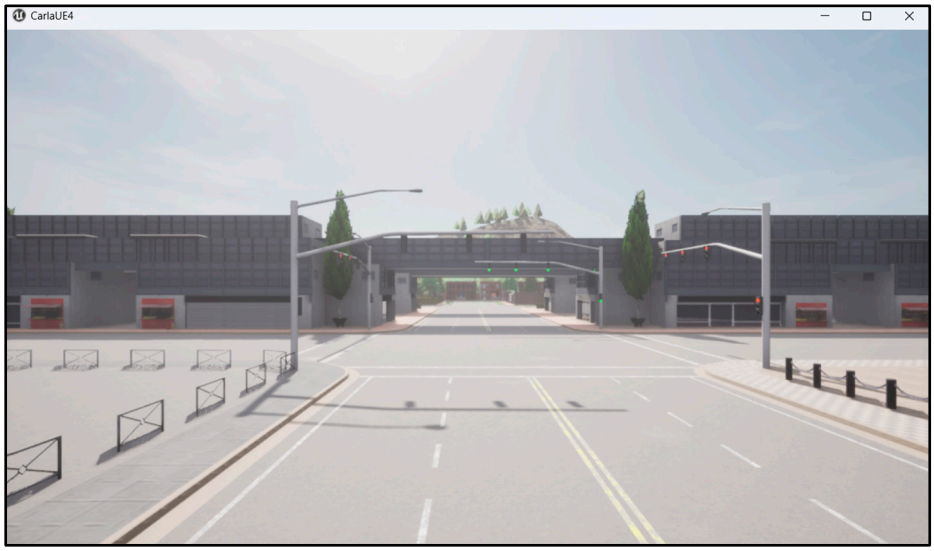

Figure 5. The CARLA highway environment (Town 05).

Using the Town 05 environment of CARLA 0.9.13, we simulated the behavior of an expert human driver to control an autonomous vehicle using the actions generated by our model. After the model was well trained, we integrated it into the actor network for RL. This approach reduces the gap between states and actions, thus accelerating the training and convergence of the model.

In the CARLA simulator, the starting and destination points were consistent, and the traffic conditions varied according to the various episodes of training in Town 05. While the autonomous vehicle was entrusted with the task of driving safely and effectively, it could turn right and left in several intersections in an urban environment.

The experimental simulation and training tests in our work were conducted using a PC HP Gaming I-5, Intel® Core ™ i5-11800h processor reconditioned 2.30 GHz 16 GB RAM, 512GB SSD Hard Drive, Nvidia GeForce RTX 3050 Graphic, 15.6 "HD LED".

To test our work, the vehicle was initially generated randomly in the starting area, and it had to follow the designated route according to the town chosen until reaching its destination; it must also avoid collisions with other vehicles in dense traffic. The environment of the town chosen also contains other fixed and mobile objects, including signaling failures, pedestrians, cyclists, motorcycles, and other vehicles.

In Figure 6, the graph represents the reward per episode for the Deep Q-Network (DQN) agent trained in CARLA over 2000 episodes. This method is used in state-of-the-art methodologies. The x-axis denotes the number of episodes, while the y-axis represents the reward value. Observing the graph, the reward values exhibit significant fluctuations and spikes throughout the episodes, indicating inconsistent learning. There is no clear trend of continuous improvement, suggesting that the agent struggles to optimize its policy effectively. Additionally, the large variance in rewards implies that the model does not achieve stable learning.

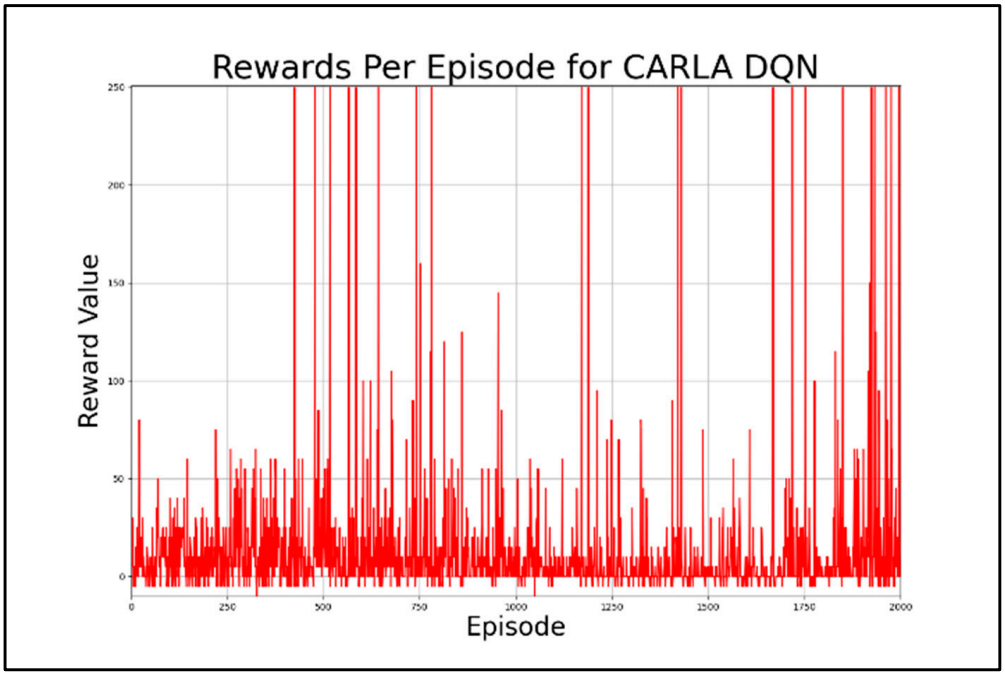

Figure 6. Representation of reward vs. episode of DQN algorithm with 2000 episodes.

DQN, in addition, has several limitations when used for autonomous vehicle (AV) training in reinforcement learning. One major issue is the lack of stability in learning, as the high variance in rewards suggests that DQN struggles with maintaining stable Q-values. Furthermore, DQN is designed for discrete action spaces, whereas AVs require continuous control over acceleration, braking, and steering. Discretizing these actions limits fine control and reduces overall performance. Another challenge is the exploration versus exploitation trade-off; DQN often suffers from insufficient exploration, leading to suboptimal policies.

Moreover, the algorithm is highly sensitive to hyperparameter tuning, such as learning rate and experience replay buffer size, making training inefficient for AVs. Small changes in parameters can result in catastrophic forgetting or suboptimal learning outcomes. Another crucial limitation is DQN's inability to handle safety constraints effectively. The high fluctuations in rewards suggest frequent collisions or unsafe driving events, highlighting DQN's inadequacy for real-world AV training without additional modifications.

Figure 7 represents epsilon decay over 2000 episodes in a reinforcement learning (RL) training process, likely for a Deep Q-Network (DQN) agent. The x-axis represents the number of episodes, while the y-axis represents epsilon values.

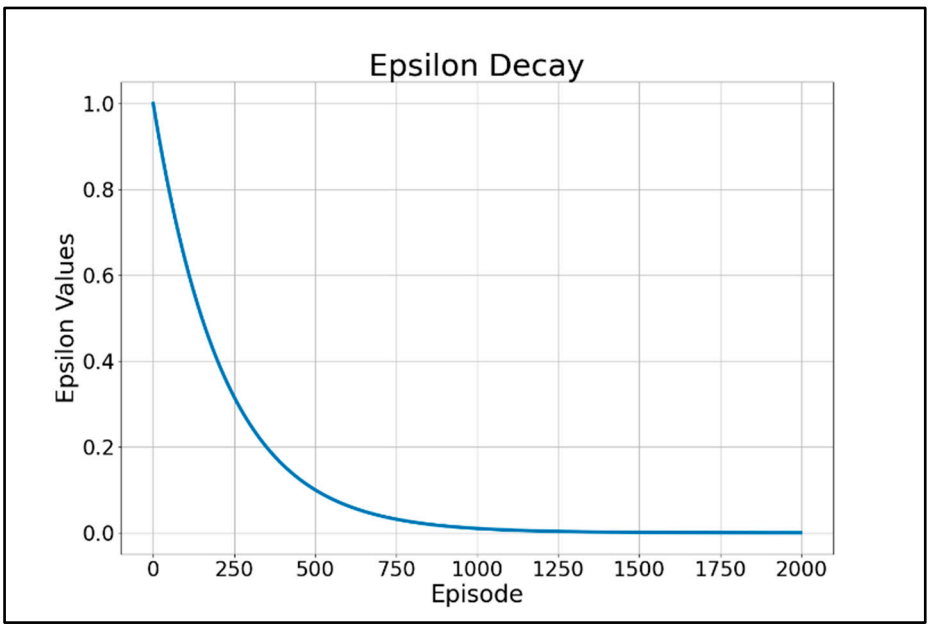

Figure 7. Representation of epsilon decay values of DQN algorithm with 2000 episodes.

Epsilon (ε) is a parameter in ε-greedy exploration used in DQN to balance exploration (trying new actions) and exploitation (choosing the best-known action). The graph shows that epsilon starts at 1.0, meaning that the agent initially explores randomly. Over time, epsilon decays exponentially, approaching near-zero values after around 1500 episodes, indicating that the agent shifts from exploration to exploitation.

Figure 8 represents the reward per episode for a reinforcement learning (RL) agent trained in CARLA over 2000 episodes. The x-axis represents the number of episodes, while the y-axis represents the reward value obtained by the agent in each episode.

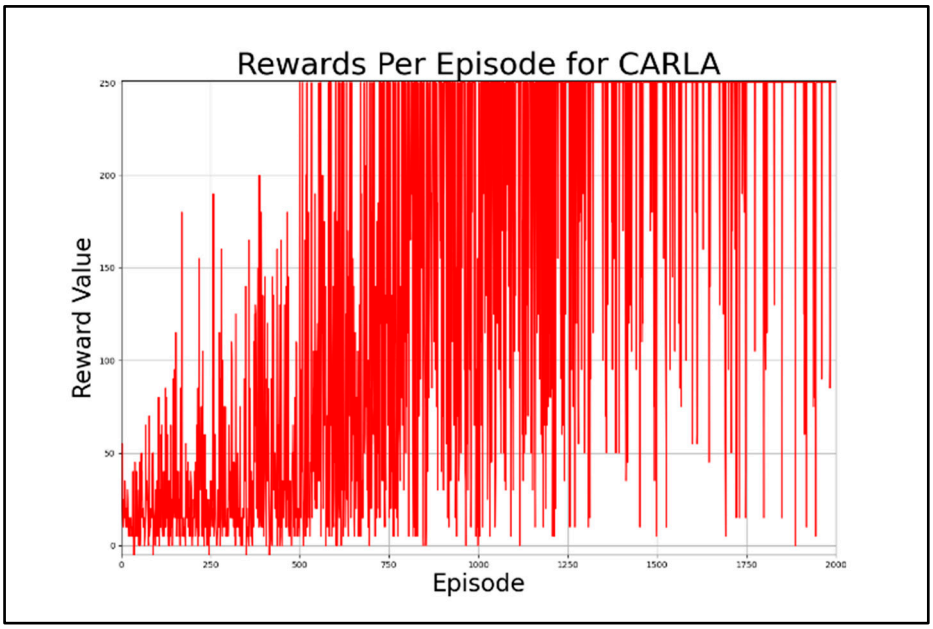

Figure 8. Representation of reward vs. episode of proposed DDQN algorithm with 2000 episodes.

The above graph nicely shows the stability and performance of the Double Deep Q-Network (DDQN) method I n the CARLA simulator, which is used for autonomous driving tasks. The upward trend in the episode rewards graph indicates that the agent is

improving its driving skills over time. This improvement suggests that the agent is learning to stay in its lane, make the right turns, and avoid collisions more efficiently as it gains experience over episodes.

Figure 9 represents eEpsilon dDecay over 2000 episodes in a reinforcement learning (RL) training process, likely for a Double Deep Q-Network (DQN) agent. The x-axis represents the number of episodes, while the y-axis represents epsilon values.

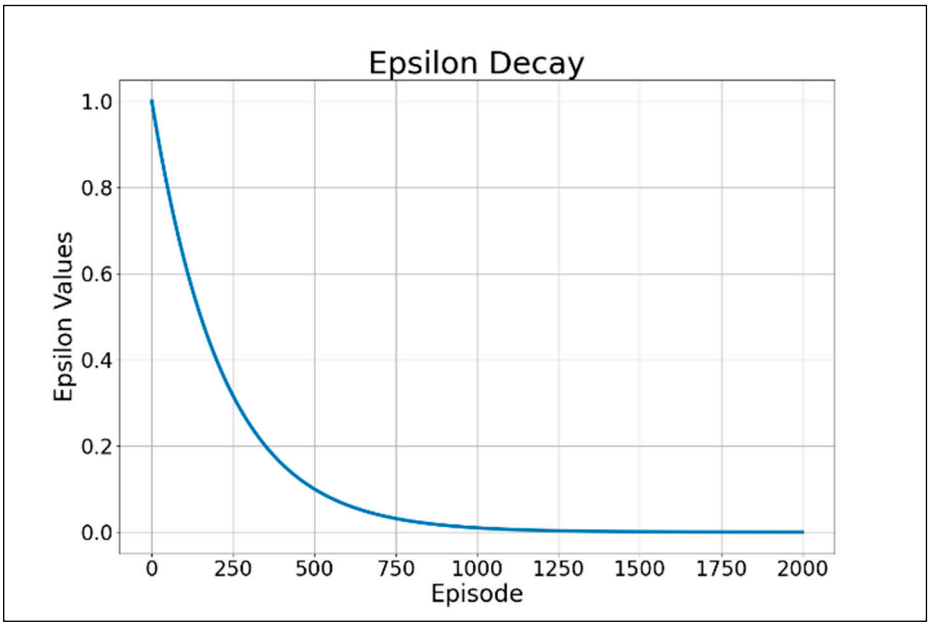

Figure 9. Representation of epsilon decay values of proposed DDQN algorithm with 2000 episodes.

7. Conclusions

In this study, we developed an autonomous driving system utilizing Deep Q-Networks with Double Q-Learning (DDQN) in the CARLA simulator. Our model, which integrates Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) layers, effectively processes sequential visual input to control a Tesla Model 3 in a complex urban environment. Through reinforcement learning, the system learned to navigate Town 05 by making informed driving decisions, including steering, accelerating, and braking. One of the key contributions of this work is the implementation of DDQN, which mitigates the overestimation bias commonly found in traditional Deep Q-Networks (DQNs). By using a separate target network and decoupling action selection from value estimation, DDQN significantly enhances training stability and improves policy performance. The combination of a replay buffer and an epsilon-greedy exploration strategy further ensured a balance between learning from past experiences and discovering new driving behaviors. Our results demonstrate that reinforcement learning can be an effective approach for autonomous driving, with the trained model exhibiting improved decision-making capabilities, reduced collision rates, and smoother driving patterns over time. These findings highlight the potential of deep reinforcement learning for developing intelligent self-driving systems that can adapt to dynamic environments without human intervention. Despite these promising outcomes, several challenges remain. This study is constrained by the limitations of simulated environments, which do not fully capture the complexities of real-world driving. Additionally, fine-tuning hyperparameters, expanding the training dataset, and incorporating more diverse driving scenarios could further enhance model robustness. Future research should explore the integration of real-world sensory data, adaptive learning techniques, and multi-agent interactions to advance the applicability of reinforcement

learning in autonomous driving. By continuing to refine these models and bridge the gap between simulation and real-world deployment, this research contributes to the broader effort of developing safer and more reliable autonomous driving technologies.

Author Contributions: Conceptualization, A.K. and M.O.; methodology, A.K.; software, A.K.; validation, A.K., M.O. and M.K.; formal analysis, A.K.; investigation, A.K.; resources, A.K.; data curation, A.K.; writing—original draft preparation, A.K.; writing—review and editing, A.K.; visualization, A.K.; supervision, M.O.; project administration, M.K.; funding acquisition, A.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Shalev-Shwartz, S.; Shammah, S.; Shashua, A. Safe, Multi-Agent, Reinforcement Learning for Autonomous Driving. *arXiv* **2016**, arXiv:1610.03295.
2. Haklidir, M.; Temeltas, H. Autonomous Driving Systems for Decision-Making Under Uncertainty Using Deep Reinforcement Learning. In Proceedings of the 2022 30th Signal Processing and Communications Applications Conference (SIU), Safranbolu, Turkey, 15–18 May 2022; pp. 1–4.
3. Qian, Z.; Guo, P.; Wang, Y.; Xiao, F. Ethical and Moral Decision-Making for Self-Driving Cars Based on Deep Reinforcement Learning. *J. Intell. Fuzzy Syst.* **2023**, *45*, 5523–5540. [CrossRef]
4. Sallab, A.E.; Abdou, M.; Perot, E.; Yogamani, S.K. Deep Reinforcement Learning Framework for Aut onomous Driving. *arXiv* **2017**, arXiv:1704.02532.
5. Kuutti, S.; Bowden, R.; Jin, Y.; Barber, P.; Fallah, S. A Survey of Deep Learning Applications to Autonomous Vehicle Control. *IEEE Trans. Intell. Transp. Syst.* **2020**, *22*, 712–733. [CrossRef]
6. Liu, Z.; Cai, Y.; Wang, H.; Chen, L.; Gao, H.; Jia, Y.; Li, Y. Robust Target Recognition and Tracking of Self-Driving Cars with Radar and Camera Information Fusion Under Severe Weather Conditions. *IEEE Trans. Intell. Transp. Syst.* **2021**, *23*, 6640–6653. [CrossRef]
7. Hoel, C.; Wolff, K.; Laine, L. Automated Speed and Lane Change Decision Making Using Deep Reinforcement Learning. In Proceedings of the 2018 21st International Conference on Intelligent Transportation Systems (ITSC), Maui, HI, USA, 4–7 November 2018; pp. 2148–2155.
8. Ronecker, M.P.; Zhu, Y. Deep Q-Network Based Decision Making for Autonomous Driving. In Proceedings of the 2019 3rd International Conference on Robotics and Automation Sciences (ICRAS), Wuhan, China, 1–3 June 2019; pp. 154–160.
9. Elallid, B.B.; Benamar, N.; Mrani, N.; Rachidi, T. DQN-Based Reinforcement Learning for Vehicle Control of Autonomous Vehicles Interacting With Pedestrians. In Proceedings of the 2022 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), Sakheer, Bahrain, 20–21 November 2022; pp. 489–493.
10. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.A.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-Level Control Through Deep Reinforcement Learning. *Nature* **2015**, *518*, 529–533. [CrossRef]
11. Elallid, B.B.; Benamar, N.; Bagaa, M.; Hadjadj-Aoul, Y. Enhancing Autonomous Driving Navigation Using Soft Actor-Critic. *Future Internet* **2024**, *16*, 238. [CrossRef]
12. Katrakazas, C.; Quddus, M.A.; Chen, W.; Deka, L. Real-Time Motion Planning Methods for Autonomous On-Road Driving: State-of-the-Art and Future Research Directions. *Transp. Res. Part C Emerg. Technol.* **2015**, *60*, 416–442. [CrossRef]
13. Giannaros, A.; Karras, A.; Theodorakopoulos, L.; Karras, C.N.; Kranias, P.; Schizas, N.; Kalogeratos, G.; Tsolis, D. Autonomous Vehicles: Sophisticated Attacks, Safety Issues, Challenges, Open Topics, Blockchain, and Future Directions. *J. Cybersecur. Priv.* **2023**, *3*, 493–543. [CrossRef]
14. Pérez-Gil, Ó.; Barea, R.; López-Guillén, E.; Bergasa, L.M.; Gómez-Huélamo, C.; Gutiérrez, R.; Diaz-Diaz, A. Deep Reinforcement Learning Based Control for Autonomous Vehicles in CARLA. *Multimed. Tools Appl.* **2022**, *81*, 3553–3576. [CrossRef]
15. Thompson, C.R.; Talla, R.R.; Gummadi, J.C.; Kamisetty, A. Reinforcement Learning Techniques for Autonomous Robotics. *Asian J. Appl. Sci. Eng.* **2019**, *8*, 85–96. [CrossRef]
16. Hu, D.; Huang, C.; Wu, J.; Gao, H. Pre-Trained Transformer-Enabled Strategies with Human-Guided Fine-Tuning for End-to-End Navigation of Autonomous Vehicles. *arXiv* **2024**, arXiv:2402.12666.
17. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; MIT Press: Cambridge, MA, USA, 1998.

18. Georgeon, O.L.; Casado, R.C.; Matignon, L. Modeling Biological Agents Beyond the Reinforcement-Learning Paradigm. *Biologically Inspired Cogn. Archit.* **2015**, *71*, 17–22. [CrossRef]
19. Rizehvandi, A.; Azadi, S.; Eichberger, A. Decision-Making Policy for Autonomous Vehicles on Highways Using Deep Reinforcement Learning (DRL) Method. *Automation* **2024**, *5*, 564. [CrossRef]
20. Chen, Y.; Ji, C.; Cai, Y.; Yan, T.; Su, B. Deep Reinforcement Learning in Autonomous Car Path Planning and Control: A Survey. *arXiv* **2024**, arXiv:2404.00340.
21. Zhang, Y.; Sun, P.; Yin, Y.; Lin, L.; Wang, X. Human-Like Autonomous Vehicle Speed Control by Deep Reinforcement Learning with Double Q-Learning. In Proceedings of the 2018 IEEE Intelligent Vehicles Symposium (IV), Changshu, China, 26–30 June 2018; pp. 1251–1256.
22. Doe, J.; Smith, A.; Lee, B. End-to-End Autonomous Driving Through Dueling Double Deep Q-Network. *Future Transp.* **2021**, *4*, 328–337. [CrossRef]
23. Hu, H.; Chu, D.; Yin, J.; Lu, L. Double Deep Q-Networks Based Game-Theoretic Equilibrium Control of Automated Vehicles at Autonomous Intersection. *Automot. Innov.* **2024**, *7*, 571–587. [CrossRef]
24. Peng, X.; Bu, X.; Zhang, X.; Dong, M.; Ota, K. Double Broad Q-Network for Overtaking Control of Autonomous Driving. *IEEE Trans. Veh. Technol.* **2024**, 1–13. [CrossRef]
25. Qian, B.; Huang, B. Autonomous Driving Decision Algorithm Based on Hierarchical Dueling Double Deep Q-Network. In Proceedings of the 2024 2nd International Conference on Signal Processing and Intelligent Computing (SPIC), Guangzhou, China, 20–22 September 2024; pp. 390–393. [CrossRef]
26. Peng, X.; Liang, J.; Zhang, X.; Dong, M.; Ota, K.; Bu, X. LK-TDDQN: A Lane Keeping Transfer Double Deep Q Network Framework for Autonomous Vehicles. In Proceedings of the GLOBECOM 2023-2023 IEEE Global Communications Conference, Kuala Lumpur, Malaysia, 8–12 December 2023; pp. 3518–3523. [CrossRef]
27. Dosovitskiy, A.; Ros, G.; Codevilla, F.; López, A.M.; Koltun, V. CARLA: An Open Urban Driving Simulator. In Proceedings of the Conference on Robot Learning, Mountain View, CA, USA, 13–15 November 2017.
28. Li, P.X.; Kusari, A.; Leblanc, D. A Novel Traffic Simulation Framework for Testing Autonomous Vehicles Using SUMO and CARLA. *arXiv* **2021**, arXiv:2110.07111.
29. Papadakis, A.; Theodorou, T.; Mamatas, L.; Petridou, S.G. An Experimentation Environment for SDN-Based Autonomous Vehicles in Smart Cities. In Proceedings of the 2021 17th International Conference on Network and Service Management (CNSM), Izmir, Turkey, 25–29 October 2021; pp. 391–393.
30. Elallid, B.B.; Benamar, N.; Hafid, A.S.; Rachidi, T.; Mrani, N. A Comprehensive Survey on the Application of Deep and Reinforcement Learning Approaches in Autonomous Driving. *J. King Saud Univ. Comput. Inf. Sci.* **2022**, *34*, 7366–7390. [CrossRef]
31. Hossain, J. Autonomous Driving with Deep Reinforcement Learning in CARLA Simulation. *arXiv* **2023**, arXiv:2306.11217.
32. Tammewar, A.; Chaudhari, N.; Saini, B.; Venkatesh, D.; Dharahas, G.; Vora, D.R.; Patil, S.A.; Kotecha, K.V.; Alfarhood, S. Improving the Performance of Autonomous Driving Through Deep Reinforcement Learning. *Sustainability* **2023**, *15*, 13799. [CrossRef]
33. Bojarski, M.; Testa, D.W.; Dworakowski, D.; Firner, B.; Flepp, B.; Goyal, P.; Jackel, L.D.; Monfort, M.; Muller, U.; Zhang, J.; et al. End to End Learning for Self-Driving Cars. *arXiv* **2016**, arXiv:1604.07316.
34. Chen, Y.; Palanisamy, P.; Mudalige, P.W.; Muelling, K.; Dolan, J.M. Learning On-Road Visual Control for Self-Driving Vehicles With Auxiliary Tasks. In Proceedings of the 2019 IEEE Winter Conference on Applications of Computer Vision (WACV), Waikoloa, HI, USA, 7–11 January 2019; pp. 331–338.
35. Chen, L.; Hu, X.; Tang, B.; Cheng, Y. Conditional DQN-Based Motion Planning with Fuzzy Logic for Autonomous Driving. *IEEE Trans. Intell. Transp. Syst.* **2022**, *23*, 2966–2977. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.